Auton Agent Multi-Agent Syst (2026) 40:25
https://doi.org/10.1007/s10458-026-09751-x

Chat with UAV – human-UAV interaction based on large language models

Haoran Wang[1,3] · Zhuohang Chen[1,2] · Guang Li[1,2] · Bo Ma[2] · Chuanhuang Li[2]

Received: 16 October 2024 / Accepted: 22 April 2026
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2026

Abstract
The future of UAV interaction systems is evolving from engineer-driven to user-driven, aiming to replace traditional predefined Human-UAV Interaction designs. This shift focuses on enabling more personalized task planning and design, thereby achieving a higher quality of interaction experience and greater flexibility, which can be used in many fields, such as agriculture, aerial photography, logistics, and environmental monitoring. However, due to the lack of a common language between users and the UAVs, such interactions are often difficult to be achieved. The developments of Large Language Models possess the ability to understand natural languages and Robots' (UAVs') behaviors, marking the possibility of personalized Human-UAV Interaction. Recently, some HUI frameworks based on LLMs have been proposed, but they commonly suffer from difficulties in mixed task planning and execution, leading to low adaptability in complex scenarios. In this paper, we propose a novel dual-agent HUI framework. This framework constructs two independent LLM agents (a task planning agent, and an execution agent) and applies different Prompt Engineering to separately handle the understanding, planning, and execution of tasks. To verify the effectiveness and performance of the framework, we have built a task database covering four typical application scenarios of UAVs and quantified the performance of the HUI framework using three independent metrics. Meanwhile, different LLM models are selected to control the UAVs with compared performance. Our user study experimental results demonstrate that the framework improves the smoothness of HUI and the flexibility of task execution in the tasks scenario we set up, effectively meeting users' personalized needs.

Keywords Prompt engineering · UAV · LLMs · Agent · Human-UAV interaction

✉ Bo Ma
mabo@mail.zjgsu.edu.cn

[1] School of Engineering and Informatics, University of Sussex, Falmer, Brighton BN1 9RH, England

[2] School of Information and Electronic Engineering, Zhejiang Gongshang University, Hangzhou 310018, Zhejiang, China

[3] Department of Electrical and Computer Engineering, University of Auckland, Auckland 1142, New Zealand

Published online: 04 May 2026

1 Introduction

With the increasing popularity of Unmanned Aerial Vehicles (UAVs) in modern society, the complexity of Human-UAV Interaction (HUI) is also escalating. According to research by Rearch and Markets, the world's largest market research organization, the global UAVs market will reach a value of up to €53.1 billion by 2025 [1]. The Civil Aviation Administration of China published [2] that the number of registered UAVs in China had reached 1.2627 million by the end of 2023, representing a 32.2% increase from 2022. Clearly, the rapid proliferation of UAVs in human society has made the need to lower the threshold for HUI even more urgent. It is becoming more important that exploring how to control UAVs through diversified user interfaces and interaction mechanisms, and conducting in-depth research on more complex interaction designs from the perspectives of personalization, privacy protection, naturalness of interaction, security, and humanistic care [3]. This trend underscores the increasingly refined nature of HUI.

Traditional control schemes are designed to meet the predefined, fixed HUI or task planning requirements set by engineers. For example, paper [4, 5] used neural networks to recognize human gestures for the purpose of enabling HUI. Rajappa et al [6] obtained human-transmitted signals by applying external forces to the drone and allowing its sensors to recognize the applied forces. In widely used agricultural UAVs, each task is executed based on task routes and parameters set by engineers, which are suitable for static scenarios. However, such control schemes have limited capabilities in handling complex tasks and often require human intervention to meet demands. To address reliability and system manageability, classical robotics research has developed three-layer architectures [7–9], which decompose capabilities hierarchically: a task-planning layer formulates goals, a behavior-control layer sequences predefined actions, and an execution layer interfaces with hardware. These architectures perform stably in domains such as UAV safety [9] and physical HUI [7]. However, their task decomposition relies on hard-coded logic such as state machines, limiting their adaptability. For example, when a user requests "avoid birds and inspect roof cracks," such architectures would typically require manually redefining state transitions and updating control graphs—an inflexible and labor-intensive process.

To improve modularity and reusability, skill-based architectures [6, 10] encapsulate capabilities into callable modules. These frameworks support task composition by assembling existing skills, such as Dong et al.'s marine disturbance controller [10]. Nevertheless, they often demand retraining of parametric models to accommodate new skills, leading to scalability and generalization challenges.

Therefore, future HUI framework is evolving towards user-driven personalized task directions, aiming to provide high-quality HUI experiences and enhance the flexibility of task planning [11, 12]. Nevertheless, achieving user-driven personalized goals through traditional methods is challenging due to the lack of a common language between users and UAVs. This absence of a common language manifests in two aspects. Firstly, due to the complexity and abstraction of programming languages, humans often find it difficult to model and solve real-world problems using code, especially for non-programmers. Thus, even if users understand the essence of a problem, they may struggle to translate it into a format that UAVs or other automated framework can execute. Secondly, humans cannot directly communicate with algorithms, machines, or programs, as they operate based on logical rules and data inputs, rather than natural language or intuitive understanding. Consequently, users may not be able to convey specific instructions or requests in a way that UAVs can understand and respond to, which can lead to misunderstandings, errors, and ultimately, a suboptimal level of personalization and effectiveness in HUI.

Fortunately, with the emergence of ChatGPT, Large Language Models (LLMs) have developed rapidly, and the advent of Llama [13] has made it a reality for individuals to deploy LLMs locally, the emergence of LLMs points to a solution to provide a common language or a swift transform between human and machines. Trained on massive amounts of textual data, LLMs possess powerful semantic understanding capabilities, enabling them to deeply comprehend the syntax, semantics, and context of natural language. At the same time, the syntactic standards, logical consistency, and abstraction and modularization features of code allow LLMs to excel in understanding and generating code. These two characteristics make LLMs capable of serving as a direct bridge between users and UAVs, marking the possibility for users to interact directly with UAVs beyond the predefined settings of engineers. Recently, many HUI frameworks based on LLMs have been proposed to realize this vision: for example, using ChatGPT directly for task planning [14] or integrating LLMs into the UAV control process through a Python pipeline [15], leveraging LLMs to abstract code for users, thereby enabling them to organize personalized tasks more freely. Liu et al [16] and Sun [17] have integrated LLMs into the visual inspection of infrastructure in order to harness the ability of LLMs to comprehend human intentions and generate control commands. However, these single-agent LLM methods still encounter planning failures when faced with complex planning challenges. The issues leading to such failures lie in:

1. **Task Planning:** When single-agent LLM methods engage in task planning, they may erroneously integrate code or execution scripts into the plan, driven by the dual necessity of planning and execution, as illustrated in Fig. 1. This incorporation introduces errors, as the plan ideally should abstract from specific code or execution intricacies. In the context of task planning, LLMs are expected to concentrate on comprehending the task and its context, akin to a 'human' role. Conversely, during task execution, LLMs need to generate precise code or machine-readable instructions, functioning more like a 'script'. In practical scenarios, achieving a seamless transition between these 'human' and 'script' roles within a single LLM run presents a significant challenge. Consequently, when confronted with intricate planning and tasks, single LLM agent methods may encounter reduced planning efficiency or execution failures.
2. **Task Execution:** Furthermore, when using LLMs for task planning or executing independent tasks, if the complexity of the task exceeds the capabilities of the predefined high-level function library, the existing HUI frameworks based on LLMs [15] may fail to execute the task, especially for complex operations like obstacle avoidance. For UAVs, obstacle avoidance involves an intricate process, including receiving camera data, inferring surrounding obstacles, and calculating subsequent velocity vectors, which cannot be simplified into a script. Therefore, in such cases, the interaction framework must possess the ability to invoke tools to complete the task. Similarly, for task planning, when the complexity of the task surpasses the solo handling capacity of LLMs, the framework should also have the capability to call upon tools.

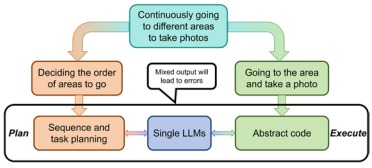


Fig. 1 Task decomposition divides a complex task into planning and execution phases, which differ for LLMs. Using one LLM-agent for both may cause errors, incorporate execution details into the plan, or conversely, insert planning elements into execution details

To address the aforementioned issues, we propose “UAV-GPT," a dual-agent framework tailored for UAV interaction. It integrates the natural language understanding capabilities of LLMs via API interfaces, enabling UAVs to “understand" human language and learn to invoke tools to tackle complex task planning and execution challenges. The framework consists of two LLM agents: a planning agent and an execution agent. To solve the task planning problem, the planning agent utilizes the natural language understanding abilities of LLMs to classify and plan tasks, accurately categorizing user needs and formulating reasonable execution plans through a predefined behavior library and discrete task reorganization. It then conveys the classified and planned tasks to the execution agent. To solve the task execution problem, execution agent will select the most suitable method and executes the corresponding operational code based on the mapping relationship between a predefined command library and the LLM, ensuring precise and efficient execution of every instruction. To enhance generalization, we integrated ROS-based control algorithms which fall within the scope of the skill base architecture, the difference is that we use skills as a supplement to the robotics framework rather than as its foundation. Meanwhile, within the classical three-layer architecture, our planning and execution agents operate within the scope of the task planning and execution layers, respectively. Crucially, the integration of LLMs significantly optimizes these layers compared to traditional implementations: it empowers the planning layer with semantic reasoning to transcend hard-coded logic, and upgrades the execution layer with code comprehension to enable dynamic rather than static tool invocation. This ensures that, unlike purely skill-based or traditional architectures, our system achieves better flexibility and adaptability.

In summary, our contributions are as follows:

- We propose a dual-agent intelligent interaction framework based on LLMs for UAV, which achieves precise classification of tasks and outputs solutions with high success rates and efficiency. Planning agent employs a predefined behavior library and discrete task reorganization to plan complex tasks reasonably, execution agent convert the solutions into precise machine language outputs.
- We introduced traditional control algorithms to the execution agent to enhance its range of applicability. The execution agent evaluates tasks through the mapping relationship between a predefined command library and LLMs, and then selects a reasonable execution method to complete the tasks.
- The user study demostrate that our framework improves HUI smoothness and task-execution flexibility, enabling better support for personalized user needs.
- The simulation and real experiments results demonstrate that, compared to the HUI framework with a single LLM agent, our framework achieves an average improvement of 60% in operational efficiency for complex tasks and a 30% increase in task execution success rate.

In the overview of Section 3, we first briefly explained the limitations of the traditional HUI for UAVs, then describe the specific architectural approach of UAV-GPT for the HUI scheme, where the interaction requirements are obtained from users through simulated requests. In Section 3.1, we construct a database related to daily interaction tasks with UAVs. We have defined four task categories for the database and provided detailed explanations for the classification rationale. Within each task category, we have designed 20 different tasks that encompass simple movements to continuous complex route planning and optimal solutions. Simultaneously, we quantify the performance of different HUI frameworks on this database

using three indicators, which allows for a clearer demonstration of the advantages of our framework. In Sections 3.2 and 3.3, we provide detailed explanations of the methods at both agents of the interaction framework. The overall architecture is primarily implemented using ERNIE 4.0 and evaluated through performance analysis.

2 Related work

2.1 Traditional human-robot interaction methods

Physical human-robot interaction (PHRI) refers to direct and intuitive communication between humans and robots without intermediary mediums, effectively conveying rich tactile feedback to humans. In the early stages of robotics, due to the lack of tactile sensing capabilities, traditional interaction methods relied on wearable devices. For example, Kim et al [18] developed an exoskeleton master arm capable of detecting torque applied by users and providing multimodal contact feedback, enabling device mobility. Another example is the vibrating motor-controlled bracelet proposed in [19], which guides robots to follow human trajectories. Haddadin et al [7] summarized the latest advancements in PHRI, while De Santis et al [8] focused on addressing the safety aspects of PHRI.

However, these interaction methods face a common limitation: the interaction distance is typically fixed and confined to a small range. Moreover, the forms of robots capable of interaction are limited, restricting effective interaction with non-anthropomorphic robots such as UAVs. To address this challenge, researchers have proposed “Teleoperation Human-Robot Interaction (THRI)." In the following, we provide an overview of this area.

Teleoperation human-robot interaction leverages the concept of indirect control, where commands are transmitted through wireless communication channels. This significantly enhances the effective distance range in human-machine interaction. For example, Peppoloni et al [20] designed a ROS-integrated interface that enables users to remotely control robots via hand gestures. Similarly, Tsetserukou et al [21] proposed a method for teleoperating robotic arms using full-body motion. Su et al [22] improved traditional teleoperation by combining 2D image transmission with a 3D virtual interface, enhancing spatial awareness during remote control. Fani et al [23] explored wearable-based interaction, offering a simpler and more intuitive experience that improves responsiveness.

These studies have greatly extended the restricted interaction distances and changed the forms of robotic agents at the execution end in physical human-robot interaction. However, they also introduced new challenges. When a user engages in remote control of a robot, it is imperative that the robot operates within a size or scale that is commensurate with the user’s environment, or adheres to predefined system scales. However, the inherent variability in environmental dimensions precludes an exact correspondence, thereby necessitating user reliance on these predetermined scales. This reliance, in turn, imposes limitations on the efficacy of remote control in practical, real-world scenarios. Hence, a pertinent question arises: Can the integration of advanced technologies, specifically LLM-based Human-Robot Interaction methods, empower the execution end (i.e., the robot) with the capacity to interpret and execute commands in a manner that facilitates more personalized responses

tailored to user-specific requirements? This exploration aims to transcend the limitations imposed by reliance on predefined scales and enhance the effectiveness of remote control in real-world applications.

Supervisory and interactive systems in UAV operations Supervisory control architectures enable strategic human oversight in UAV operations through dynamic task reconfiguration and hierarchical monitoring. Wang et al [24] mitigate adversarial environmental uncertainties by formalizing swarm task planning as a Constrained Markov Decision Process (CMDP). Their performance-function-guided SAC-Lagrangian algorithm incorporates safety boundaries (e.g., no-fly zones) into optimization objectives, achieving 96% mission success rates under electronic warfare conditions. Dong et al [10] developed a bio-inspired sliding mode controller with radial basis function neural networks (RBFNN) for marine rescue drones, using Multi-Layer Perceptron (MLP) to compensate for ocean disturbances and thruster faults. This approach ensures trajectory stability under wind and wave perturbations while reducing computational complexity, validated through Lyapunov stability proofs.

As for the ground station design, standardization and modularity are critical for scalable UAV command interfaces. KP Arnold et al [25] primarily examine the types, components, safety features, redundancy design, and future applications of UAV Ground Control Stations (GCS). E Çintaş et al [26] primarily examine how to achieve visual tracking of moving targets using UAVs equipped with low-cost hardware and novel GCS. The core objective of this system is to develop a visual tracking system capable of operating efficiently in real-time applications, particularly within complex environments featuring non-fixed perspectives and dynamic target motion.

Mission planning integrates path optimization with threat mitigation optimization with threat mitigation. Xiong et al [27] combined adaptive genetic algorithms (AGA) and sine-cosine particle swarm optimization (SCPSO) for multi-drone disaster rescue, dynamically assigning tasks while generating collision-free 3D paths around collapsed structures. Security frameworks must counter emerging threats: Castrillo et al [9] established layered protocols including MAVLink encryption, RF signal fingerprinting to detect rogue drones, and GPS spoofing countermeasures deployed in critical infrastructure protection. This dual focus on operational efficiency and electronic security addresses gaps in remote UAV supervision.

2.2 Classical robotic methods

Classical robotics architectures (e.g., Three-Layer [7–9]) Classical robotics architectures, such as the Three-Layer Architecture [7–9], ensure system reliability by structuring tasks into hierarchical layers. The task-planning layer decomposes complex goals into simpler sub-tasks, the behavior-control layer schedules predefined actions, and the execution layer controls hardware interfaces. These architectures have proven effective in domains like UAV security [9] and physical human-robot interaction [7], where reliability and stability are paramount. However, they typically rely on predefined state machines or fixed task decompositions, which can be limiting when responding to more dynamic user instructions. For example, when users provide instructions like “avoid birds and inspect roof cracks," traditional three-layer systems require manual redesign of state transitions to incorporate new tasks.

Skill-based architectures [6, 10] modularize capabilities by encapsulating them into reusable skill modules, making the system more flexible and adaptable. These frameworks allow for easy composition of skills to perform various tasks. For example, Dong et al [10] developed a marine disturbance controller that can be used to address specific environmental challenges. However, extending these frameworks with new skills often requires retraining or reconfiguring parametric models, which can be resource-intensive.

Relative to these frameworks, our planning and execution agents map to the task-planning and execution layers, respectively, with ROS based control algorithms serving as a supplement to the execution agent. The integration of LLMs improves interaction flexibility and generalization compared to these frameworks.

2.3 LLM-based human-robot interaction methods

Dialogue, as a natural mode of human communication, contributes to integrating robots into human society in the context of HRI. The emergence of LLMs with advanced natural language understanding capabilities enables them to effectively assist robots in understanding scale differences in various environments. They also facilitate the integration and transmission of tasks from the control side to the execution side (robots).

Human-robot Wang et al [28] directly utilized LLMs to generate action commands, but significant deviations existed. Mai et al [29] using Large-scale Language Model as a robotic brain to unify egocentric memory and control. Mower et al [30] extracted actions from the output of LLM and executed ROS operations/services. Ishika Singh et al in [14] proposed a code framework for task planning using ChatGPT. The core of these articles are to provide program specifications of available actions and objects in the environment to the LLMs, which then plan all possible actions. Their experiment confirmed the feasibility of this approach. Since the task processes provided by LLMs are not classified, they can only transmit simple, encapsulated low-level control functions to LLMs, inform them of specific scenarios and tasks that need to be executed, and then let LLMs process the task flows and issue commands to the execution end.

In contrast, Anis Koubaa et al. chose to integrate LLMs with ROS2. In [31], they explored the method of integrating ChatGPT into ROS2 and successfully designed a ROS2 package that could convert human language instructions into navigation commands for ROS2 robots. This package-based adaptation method can be more universally applied to other robot projects, thus having higher compatibility. However, their method only differs from [14] in terms of the path implementation. The classification of tasks and the execution of complex tasks have not been perfectly addressed.

Human-UAV Regarding the interaction between UAVs and LLMs, the challenges we face are similar to those discussed in the previous two articles. Javaid et al [32] have investigated the feasibility of combining UAVs with LLMs. With the rapid development of LLMs. Sai Vemprala et al [15] utilized ChatGPT to achieve UAV control and path planning. Regarding the issue of UAV mission planning, they also demonstrated simple autonomous obstacle avoidance behaviors controlled by ChatGPT in AirSim simulations. Their work demon-

strated the efficiency of using the ChatGPT language model for robot control. Cui et al [33] employed LLMs to plan UAV missions, utilizing a single natural language input to command multiple UAVs in both synchronous and asynchronous modes. Lykov et al [34] have achieved rapid swarm control of UAVs utilizing GPT. The described method enables intuitive orchestration of swarms of any size to achieve desired geometric shapes.

However, since they only used the UAV SDK control function (i.e., controlled by Python scripts) without integrating the ROS platform, and they only used a single LLM-agent to simultaneously handle task planning and execution, using pre-prompts for function learning in LLMs without separating planning and execution, their architecture could not accurately implement the planning and execution of long and complex tasks. Mohamed Lamine Tazir in reference [35], improved the hardware equipment and used ChatGPT to transmit control information to UAVs based on PX4/Gazebo simulations, successfully controlling the UAV's operations. This was a very promising attempt because it combined ROS and ChatGPT in their system, but issues with task classification and long-process planning still persist. If we want LLMs to demonstrate their assistive capabilities in more scenarios, such as obstacle avoidance, path planning, or longer task planning, we need multiple LLM-agents and a deeper integration of ChatGPT into ROS nodes (Table 1).

3 Methods

3.1 Overview

The traditional method for HUI with UAVs is depicted in Fig. 2. It primarily hinges on users' ability to abstract real-world tasks into coding representations and subsequently transmit these to the UAV via wireless links. Inherently, this operational procedure presents an obstacle, as not all users own the capability to translate real-world tasks into code. This is precisely why it is necessary to introduce LLMs into this process. To more concisely and clearly describe UAV-GPT, we characterize it as a converter USR2ML based on LLMs that takes users' speech/text as input and converts it into executable machine language vectors (MLV):

$$l = (l_1, l_2, l_3 ..., l_T) \tag{1}$$

Where l denotes the length of the Machine Language Vector (MLV). For each time step $t \in 1, ..., T$, the output length l_t should be kept within a reasonable range, as the UAV is

Table 1 Comparison of Methods: Here, we clearly demonstrate the advantages of our method compared to traditional physical and teleoperation HUI, as well as single-agent HUI based on LLM

HUI	Force Reflected [18]	ROS-IntegratedFramework [20]	Prog-Prompt [14]	ROS-LLM [30, 31]	Prompt Craft [15]	Words to Flight [35]	Ours
Remote Control	✗	✓	✓	✓	✓	✓	✓
Personalized Tasks	✗	✗	✓	✓	✓	✓	✓
High-Level Plan	✗	✗	✗	✗	✗	✗	✓
Tools Ability	✗	✗	✗	✗	✗	✗	✓

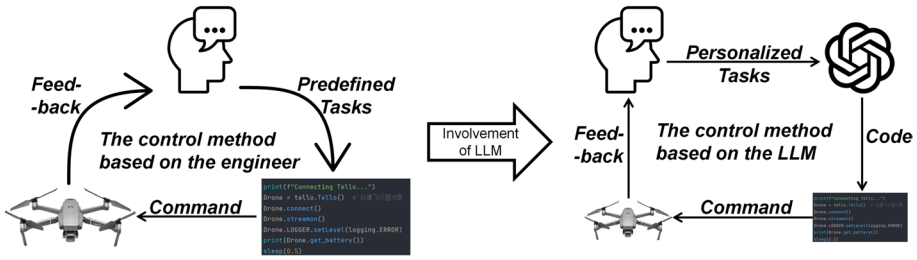


Fig. 2 Traditional teleoperation control method

required to execute the complete instruction in a single step. If the output is too long, it may result in interaction failure due to system overload or motion deviation. In particular, when the execution length exceeds a certain threshold, the UAV's trajectory may exhibit significant drift caused by cumulative errors from onboard sensors and flight controllers. Since such drift is closely tied to hardware limitations—which are beyond the scope of this study—we define a reasonable range of execution length through three points to minimize the impact of hardware-induced errors and ensure task reliability. This reasonable range is formally defined as a constraint ensuring UAVs execute complete MLVs without exceeding the system's physical or computational capacity. This constraint is determined by:

- Processor limitations (Tello SDK buffer capacity)
- Real-time communication stability (WiFi packet size thresholds)
- Safety protocols (max command chain before failsafe activation)

Experimentally, we set $l_{\min} = 3$ (e.g., takeoff→hover→land) and $l_{\max} = 7$.

When UAV-GPT outputs MLV (Machine Language Vector), we will detect its length. For outputs with a length exceeding $l_{\max}$, we consider the system's task execution as failed. The reason is that the task language received by the UAV should be within a specified range; otherwise, it may fail to execute due to hardware resource constraints. Meanwhile, for tasks with longer processes, one of the design goals of the planner is to split the overall task into multiple sub-segments that can be executed in a single run. Therefore, if the output length exceeds $l_{\max}$, it can be regarded as a failure of the planner in task division.

To achieve users' requirements, UAV-GPT first needs to classify the user's task. The application scenarios of UAVs in reality are complex and variable, and identifying the accurate task type will significantly enhance the success rate and efficiency of the agent. Next, it needs to solve the problem and output a machine language vector that meets the requirements. To achieve this goal, we propose a dual-agent architecture, as shown in Fig. 4. The first stage of UAV-GPT is a planning agent, whose task is to convert user intentions into fixed categories of tasks and reorganize the optimal solutions through a predefined behavior library and discrete task reorganization. The second stage is a machine language execution agent, whose task is to evaluate the task flow and output machine language within a reasonable range by leveraging the mapping between the LLM and the predefined instruction library.

3.2 Task classification

In the task classification framework, we adopt a two-dimensional tagging system as shown in Table 2, where the first core dimension is the **"Simple-Complex"** continuum. This dimension uses the following two sub-dimensions to characterize:

- **State Space Complexity** S_c: It is jointly determined by the number of monitoring points $p \in C_t$ and the range of dangerous areas $d \in A$, where the C_t is the content of task and A is the space of task in System Prompt(which is the preset system knowledge of the current task scene), reflecting the amount of perceptual information that needs to be focused on for the task.
- **Motion Space Complexity** M_c: It is determined by the length $l \in C_t$ of the action sequence, reflecting the reasoning depth required for the large language model to solve the problem.

In human-computer interaction scenarios, an increase in the number of task monitoring points p and the range of dangerous areas d means more optional paths and execution conditions, which directly exacerbates the complexity of the task [36]. Meanwhile, cognitive load theory [37] indicates that a larger number of monitoring points and an expanded range of dangerous areas will cause LLMs to process richer context of task C_t and perform more intermediate reasoning steps before executing the task, ultimately increasing the complexity of the task.

Next, we define the overall task complexity as a linear combination:

$$\mathcal{C}_{\text{task}} = \alpha \cdot S_c + \beta \cdot M_c, \tag{2}$$

here, $S_c = \gamma_p \cdot p + \gamma_d \cdot d$, which is the state space complexity determined by the number of monitoring points p and the range of dangerous areas d. $M_c = \gamma_a \cdot l$, which is the motion space complexity determined by the action sequence length l required by the task. Figure 3 shows how we decide these values during a task.

Note that our tasks are strictly limited to a 50×50×50 meters volume. This prevents conflicts where simple tasks requiring long-distance execution receive an inaccurately low $\mathcal{C}_{task}$ relative to their actual complexity. Instructions must also be explicit and non-autonomous to ensure that p and q remain computable, even when their values are zero. For example, high-autonomy tasks "search for a specific location" are excluded due to the infeasibility of defining p. In contrast, commands "take off and move forward 5 meters" remain valid for setting p and q to zero simply shifts the final complexity $\mathcal{C}_{task}$ to action sequence length l without compromising classification accuracy.

Table 2 Classification Tags

	Simple	Complex
Independent	Move forward 5 meters and take a picture	Move forward 5 meters then take pictures for kitchen and two bedrooms
Tool-assisted	Move forward 5 meters and avoid obstacles in time	Move forward 5 meters then take pictures for the kitchen and two bedrooms and avoid obstacles in time

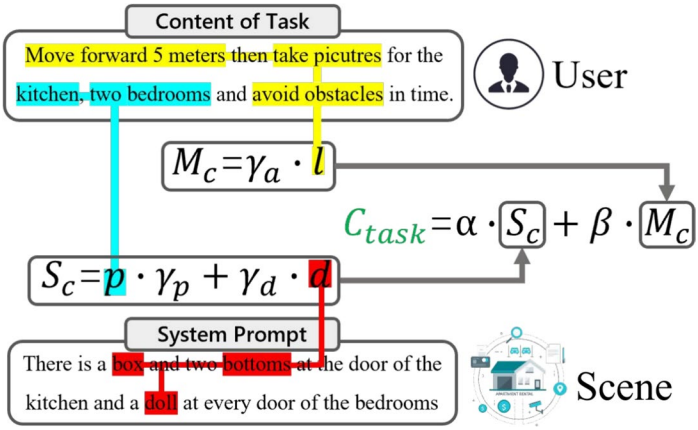


Fig. 3 The red parts represent dangerous areas, the yellow parts represent action sequences, and the blue parts represent monitoring points

The coefficients α and β control the balance between perception and reasoning burdens, while γ_p, γ_d, and γ_a serve as unit scaling factors. Based on this score, we define a two-level classification rule:

$$\text{Task Type} = \begin{cases} \text{Simple}, & \text{if } C_{\text{task}} \leq \theta \\ \text{Complex}, & \text{if } C_{\text{task}} > \theta \end{cases} \tag{3}$$

the threshold θ is an empirical or learned parameter that reflects the separation boundary in the "simple-complex" continuum. To determine the coefficients γ_p,γ_d,γ_l and θ involved in the task complexity Formulation (2), we utilize two expert-annotated datasets with high/low complexity task labels: CLAD [38] and BRMData [39]. By analyzing the p, d, and l within these datasets and correlating them with the provided task complexity labels, we estimate the relative weights of each factor. In this work, we empirically set the balance coefficients as $\alpha = \beta = 0.5$, assuming equal contributions from state space complexity and motion space complexity for simplicity.

For the second dimension of Table 2, **"Independent vs. Tool-assisted"**, we adopt a knowledge-based decision mechanism. Specifically, the system incorporates two types of prior knowledge:

- **Internet Knowledge** refers to general, preloaded knowledge that forms the foundational information base of the robot;
- **System Knowledge** refers to task-specific knowledge dynamically generated or loaded for the current interaction, including contextual and environmental information relevant to the scenario.

During execution, we extract **task-related keywords** K from the input instruction C_t: $K = \{k_1, k_2, ..., k_n\}$ where $k_i \in \text{Keywords}(C_t)$, and match them against the keywords present in system knowledge and Internet knowledge: $K_{total} = K_{sys} \cup K_{net}$. If the extracted keywords can be fully matched with the system knowledge, the task is classified as Independent,

as it can be handled with existing system information: $\forall k \in K, k \in K_{total} \Rightarrow \text{Independent}$. Otherwise, if the keywords require additional unknown tools or capabilities beyond the knowledge bases, the task is labeled as Tool-Assisted, implying dependence on external modules or resources: $\exists k \in K, k \notin K_{total} \Rightarrow \text{Tool} - \text{Assisted}$.

This classification approach allows the system to dynamically determine task autonomy based on the semantic alignment between user instructions and available knowledge, thereby bridging high-level language understanding with execution-level resource requirements. The generalizability of the proposed framework is currently bounded by the calibration dataset used during the coefficient learning phase. Specifically, the classification of UAV interaction tasks relies on the coefficient ranges derived from this dataset. Therefore, the broader and more diverse the calibration data, the wider the range of tasks the system can effectively support. This design ensures a balance between interpretability and scalability, allowing the framework to gradually extend its capability as more representative task data becomes available. In our implementation, we have made efforts to enrich the calibration dataset and ensure that the resulting classification coefficients are both reasonable and robust. By combining these two dimensions, we derive four distinct task types: **SI,ST,CI,CT**.

The planning agent of the UAV-GPT framework is responsible for comprehending user intent and accurately classifying tasks into the first dimension. The execution agent, meanwhile, need to decide if the task is independent, and choose the right method to do the task through the 2-dimensional classification, translates these classified tasks and plans into machine language for ultimate execution. In designing the framework, we leverage three key capabilities of LLMs: Firstly, converting real-world task descriptions into code tasks. Secondly, identifying task types. Thirdly, assessing tasks and selecting optimal implementation approaches through LLM's reasoning abilities. To evaluate the performance of the two agents within the UAV-GPT framework, we employed three performance metrics: Intent Recognition Accuracy (IRA), Task Execution Success Rate (ESR), and UAV Energy Consumption (UEC) for qualitative testing.

3.3 The design of planning agent

The core function of the LLM-based task planning agent is to receive user voice or text instructions and perform complexity classification and planning accordingly Fig. 4. To support this process, a two-dimensional task categorization system is introduced into the LLM, along with clearly defined classification rules. This enables the agent to interpret diverse natural language expressions of the same task and map them to appropriate structured plans. Specifically, the four task types include: simple independent tasks (e.g., "move forward 10 meters and then ascend"), simple tool-assisted tasks (e.g., "move forward 10 meters while avoiding obstacles in real-time"), complex independent tasks (e.g., "go to the kitchen, bathroom, and living room to take photos and return"), and complex tool-assisted tasks (e.g., "go to the kitchen, bathroom, and living room to take photos and return, avoiding obstacles in real-time during the journey").

To enhance the recognition capability of the interaction framework, we have designed a specific guidance path for the planning agent. This path requires the LLM to extract key information from user inputs: Scene Keywords and Task Actions. The LLM infers the possibility of task planning scenarios based on Scene Keywords and differentiates execution

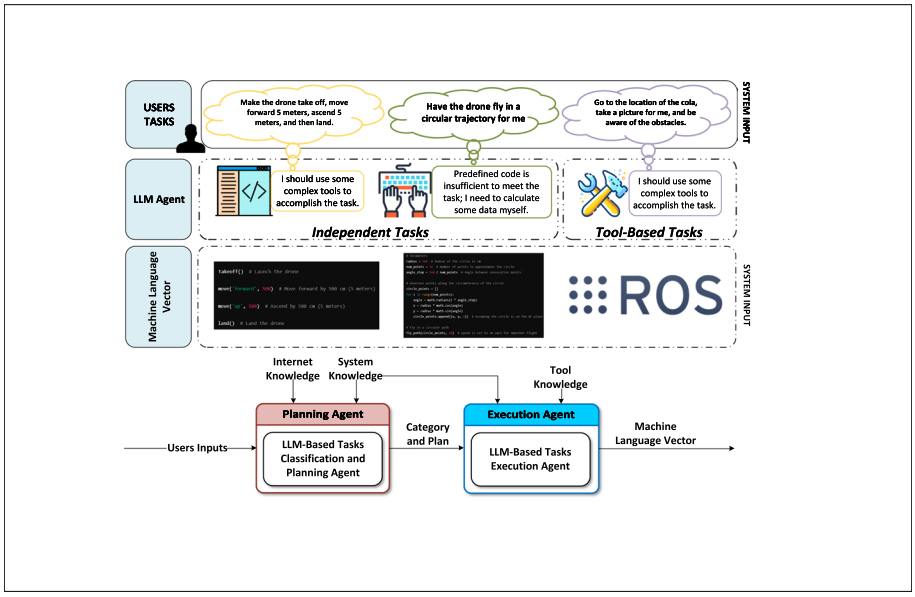


Fig. 4 This is the dual-agent architecture for user's requests to machine language vector

methods through Task Actions. These two pieces of information correspond to the first and second dimensions of task classification, respectively. This guidance scheme enables the LLM to classify the two indicators separately, significantly improving classification accuracy.

Regarding task planning, there are notable differences between the planning and execution agents. The planning agent addresses the one-dimensional range issue in task classification, employing distinct planning approaches for tasks of varying complexities. For simple tasks, the LLM describes the action sequence the UAV needs to perform in precise natural language. For complex tasks, the LLM utilizes tools such as pre-trained neural networks or fine-tuned information to plan for optimal success rates and low energy consumption. In summary, the design of the planning agent encompasses selecting an LLM model (in this case, qianwen), utilizing prompts to design the guidance structure, and creating a carefully crafted 2D task classification list with well-structured context. This setup enables the planning agent to effectively accomplish classification and planning tasks within the UAV-GPT framework. To evaluate the performance of the planning agent in terms of IRA and UEC , we have constructed a UAV-GPT database for experimental testing.

3.4 The design of execution agent

A common approach to tackling task execution for the UAV is to rely solely on an LLM agent, which utilizes LLMs such as qianwen or LLama3 to convert naturally described task workflows into machine language (code) comprehensible by the UAV, subsequently transmitting it via remote communication facilities. For instance, the CaP Framework [40] adopts this method. CaP is a robot-centric language model, generates programmatic representations capable of yielding reactive policies (like impedance controllers) and waypoint-

based strategies (e.g., vision-based grasping and placing, trajectory-based control). The core of the CaP approach lies in linking classical logical structures, incorporating third-party libraries (e.g., NumPy, Shapely) for arithmetic operations, and allowing LLMs to receive commands and autonomously recombine API calls to produce novel strategy code. While promising, this method is constrained by its execution format, generating script-based strategies for robots to execute, limiting real-time interaction with robots during task execution and thus insufficient for tasks like real-time obstacle avoidance.

Algorithm 1 Main Loop for UAV-GPT, p is monitoring points, d is range of dangerous areas and l is the length of action sequence in user's input.

Initialize Planner Agent
Load AirSim basic and system prompt
while true **do**
  `input` ← GET{USER&PRE}INPUT
  if `input` = !quit or `input` = !exit **then**
    EXITPROGRAM
  else if `input` = !clear **then**
    CLEARSCREEN
  else
    $(p, d, l) = LLM_{plan}$(`input`)
    get `complexity` $\mathcal{C}_{\text{task}}$ based on (2)
    if $\mathcal{C}_{\text{task}} > \theta$ **then**
      `taskType1` = `Complex`
    else
      `taskType1` = `Simple`
    end if
    `Plan` = $LLM_{plan}(C_t$, `taskType1`)
    OUTPUTPLAN(`Plan`)
  end if
end while

Initialize Execution Agent
Load Tool basic and system prompt
while waiting for **Task Type1** and **Plan do**
  `input1` ← SYS- KONWLEDGE
  `input2` ← IN- KONWLEDGE
  `keywords` = $LLM_{execut}(C_t)$
  `Pre_K` = `input1+input2`
  if `keywords==Pre_k` **then**
    `taskType2` = `Independent`
  else
    `taskType2` = `Tool_Based`
  end if
  if `TaskType2` = `Independent` **then**
    `action` = LLM_{execut}(`Plan`)
    EXECUTEPLAN(LLM)
  else
    `action` = LLM_{execut}(`Plan`)
    EXECUTEPLAN(LLM and Tool)
  end if
end while

Therefore, building upon our execution agent architecture, we preserved the LLM's capability to translate natural language into machine language while introducing tool invocation abilities. As illustrated in Algorithm 1, after the user provides an instruction, the system first computes the task complexity based on monitoring points p, danger range d and input length l. The Planner Agent then will determine which kind of task type it is and generates a corresponding action plan. For the the Execution Agent, it is equipped with a neural network-based RGB image recognition framework integrated with egoplanner for monocular real-time obstacle avoidance (i.e., path planning functionality) in Fig. 5 and it will decide how to execute the task based on the plan and task type from the planner agent.

We encapsulate the API-based LLM invocation behavior into a lightweight ROS package. Within the ROS system, the *llm* node publishes instruction information from the API, which is subscribed to by the EgoPlanner algorithm to determine whether to initiate the path planning process, this framework enables monocular real-time obstacle avoidance (i.e., path planning functionality). In the LLM prompt for the execution agent, we instruct the LLM to employ pure agent functionality for independent tasks, converting naturally described tasks into machine language (code), and to invoke predefined tools through ROS for tool-assisted tasks Fig. 4. To evaluate the performance of the execution agent in terms of Task Execution

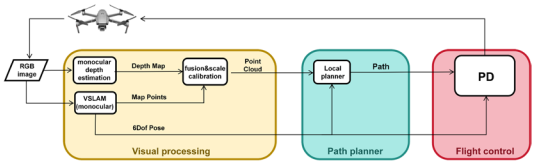


Fig. 5 The UAV uses ROS with multiple algorithms to perform tasks: capturing an RGB image, processing it for depth estimation and 3D mapping, planning a path, and adjusting flight with a controller

Success Rate (ESR) and UAV Energy Consumption (UEC), we will conduct experimental tests using the constructed UAV-GPT database.

For example, regarding a task “Move forward 5 meters then take pictures of the kitchen and two bedrooms and avoid obstacles in time", assuming the system prompt indicates that there are obstacles at the doorways of the kitchen and the bedrooms, the execution process of the system is as follows:

1. **Knowledge Matching & Task Classification**

   - **Keyword Extraction:**
     - Actions: move forward, take pictures, avoid obstacles
     - Targets: kitchen, bedrooms
   - **Knowledge Base Comparison:**
     - *System Knowledge:* Predefined navigation parameters (e.g., egoplanner node), camera APIs.
     - *Internet Knowledge:* UAV Control APIs.
   - **Decision Mechanism:**
     - Tool-Assisted Task : avoid obstacles requires real-time obstacle data → Triggers external collision-avoidance module.

2. **Executable File Generation**

   - execute_egoplanner(target="kitchen", obstacle_data=env_obstacles)
   - activate_camera(camera_type="RGB")

4 Experiment

This section presents the experimental results of the UAV-GPT framework, aiming to validate its practicality through the operations of IRA, ESR, and UEC. Meanwhile, we have set up experimental scenarios in both simulation and real-world environments. The simulation scenario is built using the Airsim simulator and the Ue4 engine, while the real-world scenario is established with a laptop equipped with RTX3080 and a Tello UAV.

4.1 Dataset and evaluation metrics

To evaluate the performance of our framework in complex tasks, we generated various requests that combine simple tasks Fig. 6, encompassing multiple decision types related to UAV control behaviors in Table 2. Specifically, our dataset includes different experimental backgrounds such as moving forward, taking photos, searching for objects, indoor navigation, outdoor inspection, etc., as well as specific experimental tasks based on these backgrounds. Additionally, these tasks are expressed explicitly and implicitly. In explicit expressions, tasks clearly require the UAV to go somewhere and perform certain tasks, such as the task types proposed in Task List 1. However, implicit expressions do not explicitly state the purpose but use alternative phrasing to allow the LLM to understand the specific details of the task, for example, “I want to eat watermelon, but I don’t know where it is." The dataset contains a total of 160 task types for calculating $\mathcal{C}_{\text{task}}$, with each two-dimensional label containing 40 tasks. By comparing the task classification labels output by the planning end with the labels of the tasks themselves, IRA can serve as an effective measure of accuracy.

Furthermore, each task in the dataset has corresponding completion criterias, which are used to compare with the output of UAV-GPT’s solutions to determine whether the agent has successfully addressed the users’ needs. IRA measures the proportion of tasks accurately classified by the planning agent within a hypothetical task database, with human classification accuracy set as the 100% benchmark. ESR reflects the ratio of successful task executions to total executions under the same task classification and plan inputs. Finally, UEC indirectly indicates power consumption by measuring the flight duration of the UAV during various task executions, we compare the UEC levels under the same task to indicate the energy-saving performance. Since the flight speed of the UAV is fixed in this scenario, we use flight time to estimate the UAV’s energy consumption, enabling more precise data information.

4.2 Prompt engineering

We briefly present different pre-prompt structures in Fig. 7

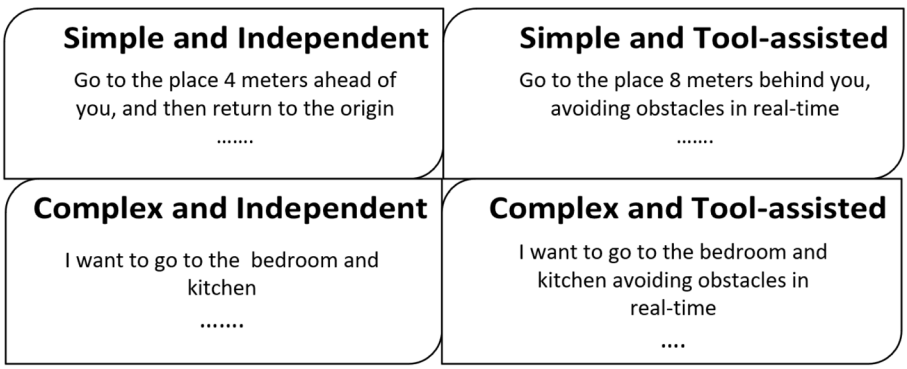


Fig. 6 Datasets, the task here are only symbolic; in reality, the task classification is determined by the empirical parameters derived from the expert-annotated dataset

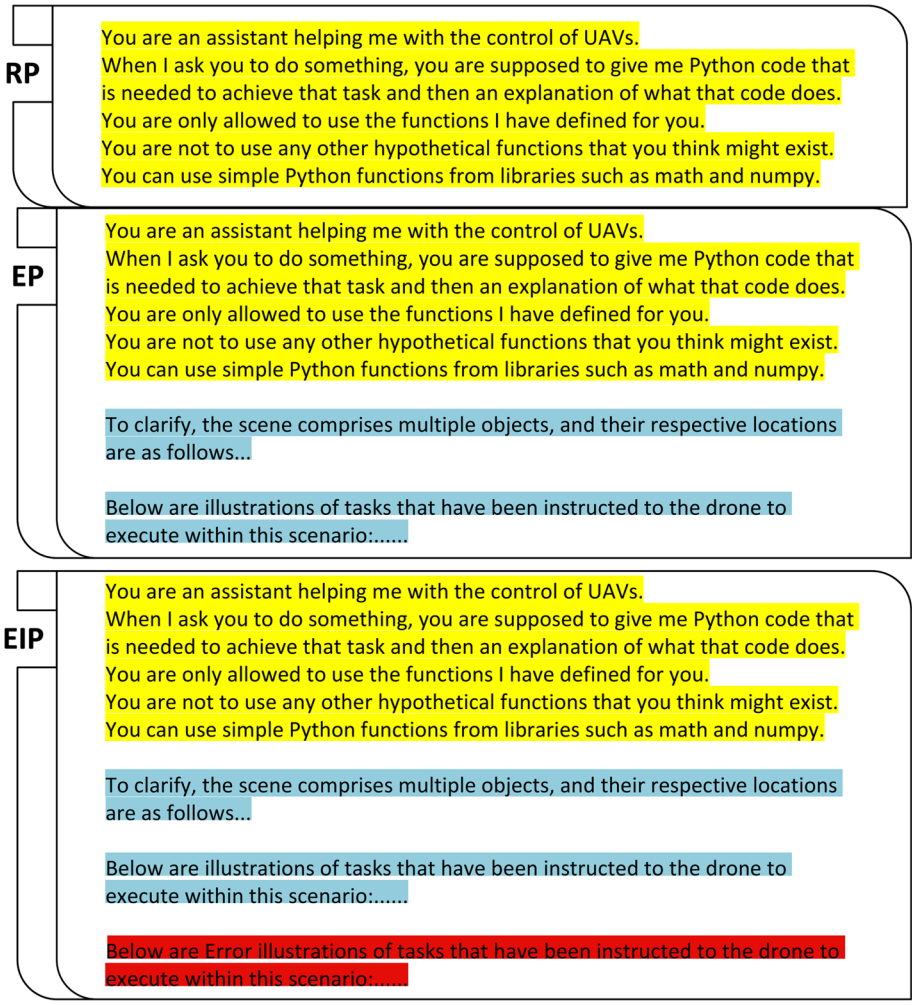


Fig. 7 Prompts

General prompt (RP) In general prompts, the LLM is informed of the role it needs to play, the tasks it needs to perform, and the necessary location information for executing those tasks. However, it is not provided with any background knowledge related to the tasks or additional information about the current scenario. It needs to plan and act based on very limited cognition.

Contextual prompt (CP) Based on general prompts, with contextual prompts, the LLM is provided with multiple task templates and corresponding outcomes for learning. Additionally, it is given extra information related to the input task, such as the coordinates of all objects in the scenario and the locations of potential obstacles. This information greatly enhances the LLM's ability to understand the scenario. We hope that through this approach, the LLM can accurately understand the user's needs and generate solutions.

Iterative prompt for errors (EIP) Based on the first two types of prompts, we simplify the failed solutions output by the LLM and use them as prompt texts in the pre-prompting stage. This can further refine the system prompts and improve its performance. By standardizing the structure and content of the prompts, this method can increase the number of task templates and outcomes (negative templates) in the prompts, allowing the LLM to anticipate more potential task scenario solutions and ensure more accurate and reliable classification and solution outputs.

4.3 Experimental results

We used ERNIE-4.0 and GPT-4o as the base models and set the randomness of the models to 0 to eliminate possible random responses and ensure consistent classification of requests by the LLM. At the same time, we used GPT-3 and Llama3 70B as controls to enhance the persuasiveness of the experiments and verify the versatility of the framework. Firstly, Fig. 8 clearly reveals the differences in task planning and execution strategies between our framework and the traditional HUI framework based on LLMs when it comes to executing tool-assisted tasks. Due to the incorporation of a ROS-based obstacle avoidance tool, UAV-GPT will plan a smoother route (b) when performing such tasks compared to the traditional framework. Moreover, because the dual-end agent can prevent LLMs from confusing planning with execution output, which may lead to irrational planning, UAV-GPT will devise a more reasonable and time-saving route when confronted with complex tasks (c). Therefore, we can qualitatively conclude that UAV-GPT exhibits better performance than the traditional LLM framework when dealing with tool-assisted and complex tasks. We repeated the experiments in Fig. 9 using the Tello UAV in real-world scenarios to further validate the reliability of our conclusions in Fig. 12.

Next, in order to comprehensively evaluate the performance of UAV-GPT, we will adopt three indicators: IRA, ESR, and UEC, as the testing standards to quantitatively assess the performance of our framework:

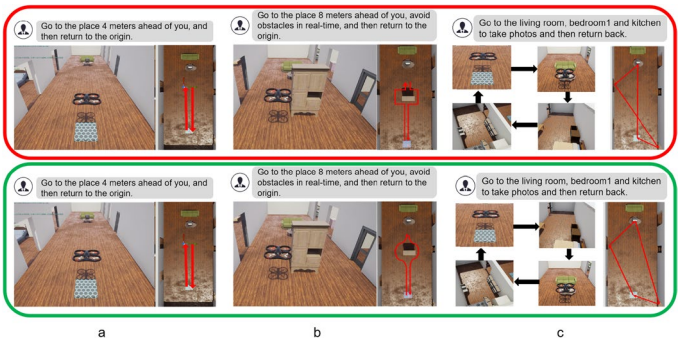


Fig. 8 The single-ended planning framework (red box) and the UAV-GPT method (green box) exhibit significant differences in strategies when executing ST and CI tasks. UAV-GPT is more reasonable in execution

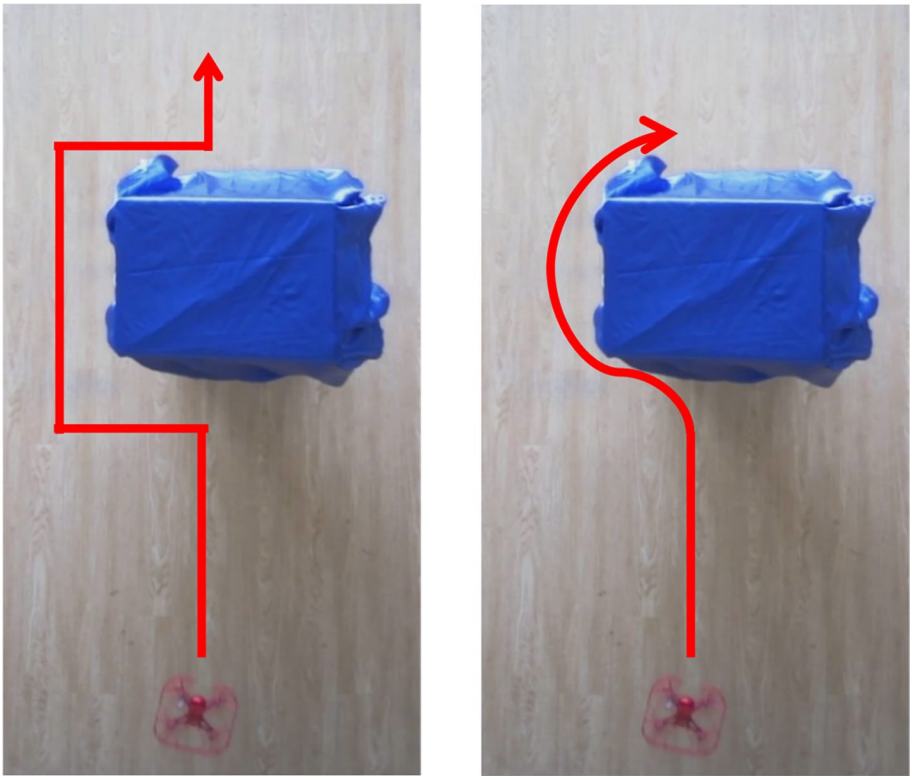


Fig. 9 This is the same tasks that we completed in real-world scenarios using the Tello drone

4.3.1 Intent recognition accuracy(IRA)

Figure 10 demonstrates the influence of employing various types of prefix prompts (EIP) on the success rate of task classification by UAV-GPT when confronted with tasks of diverse classification labels. Here, we uniformly use ERNIE-4.0 for testing to avoid the influence of different foundational LLMs on the success rate of prefix prompts in classification, as it provides a good balance between latency and accuracy and is therefore well-suited for our experiments. The findings reveal that EIP achieves the highest IRA metric among the three distinct types of prefix prompts. Specifically, in the CI and CT tasks, compared to RP, the success rate of EIP prompts has increased by an average of 25%. When compared to EP, in the CT task, there is a 23% improvement, which stems from the incorporation of environmental information and past misclassifications. Consequently, in subsequent experiments, we will adopt EIP prompts as the type of prefix prompt to minimize the influence of prefix prompts on the experimental outcomes.

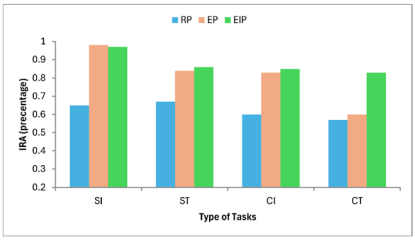


Fig. 10 The performance comparison of IRA with different prompt

4.3.2 Task execution success rate (ESR)

Figure 11 demonstrates the differences in the response of three interaction frameworks to the ESR metric when faced with tasks of different classification labels, based on the premise of EIP. The results show that for simple independent tasks and simple tool tasks, there is no significant difference in success rate between the three methods; however, when faced with complex tasks, whether they are complex-simple tasks or complex-independent tasks, the introduction of the dual-end planner significantly increases the success rate of UAV-GPT compared to the other two methods (PromptCraft [15] and CaP [40], representing single-agent LLM SOTA baselines). Additionally, 'Ours' employs ERNIE-4.0 as the base LLM model. In the case of CI task classification, the efficiency of UAV-GPT improves by 34% compared to the other two methods, and by 61% in CT tasks. We attribute this result to the improvement in task classification efficiency and the addition of the planning end. To further explore the reasons, we conducted additional experiments on UAV-GPT, integrating the planning end and execution end into a single planner, and then performing task classification and execution. The results in Fig. 11 confirm our hypothesis: for simple category tasks, there is no significant difference in success rate between single-agent planning and dual-agent planning; however, for complex category tasks, the advantage of dual-agent planning is evident.

We have also designed an experiment to verify whether the involvement of external tools can improve the success rate of task execution. We constructed a dataset consisting of 20 independent "simple-tool" tasks and used UAV-GPT to execute these tasks under two different settings: one where the system can normally call external tools, keeping the task flow consistent with regular operation; the other where external tool calls are artificially prohibited, meaning that even if the system has identified the task as a "simple-tool", it cannot actually schedule the relevant modules for execution. Through this comparative experiment, we aim to further verify the key role of external tools in task execution. The results in Fig. 12 clearly show that the system's execution capability is significantly affected in the absence of external tool support, thereby highlighting the importance of tool involvement for specific task types.

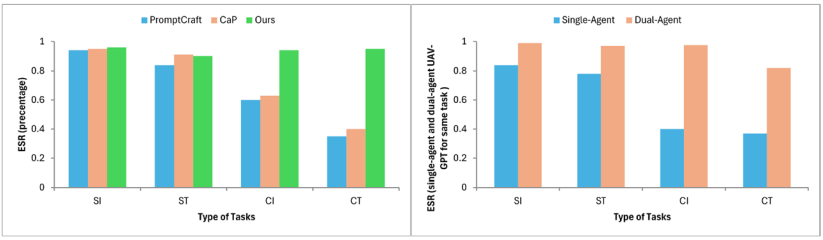


Fig. 11 The performance comparison of ESR with different HUI frameworks

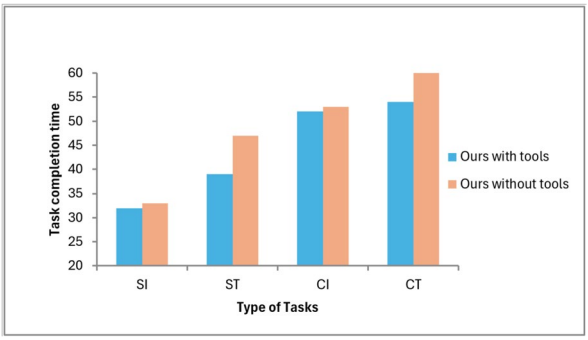


Fig. 12 The difference between an Execution Agent using external tools and not using external tools

4.3.3 UAV energy consumption (UEC)

Although UAV-GPT demonstrates a high completion rate across tasks with four classification labels, the efficiency of UAV task execution is also a crucial indicator that cannot be overlooked in practical applications. If a UAV can accomplish a task but takes an excessively long time, it is clearly unreasonable. Therefore, we next selected ST and CI category tasks for testing (we picked four ST and CI tasks from the first two indicator tests that all three interaction frameworks could complete), and let the three frameworks output solutions and execute them respectively. Figure 13 show that in the face of simple tool-based and complex independent category tasks, our method exhibits a significant advantage over the other two pure scripting and single-planning-end methods. To control variables and avoid confounding factors, we still used the EIP prompt—the best performer under the IRA metric—and fixed ERNIE-4.0 as the LLM base for all tested method. The PX4 Autopilot was also consistently used as the simulator UAV controller for three tested methods to minimize system-level interference.

In real-world scenarios, the power consumption of a drone before and after flight can be calculated from the battery capacity before and after the flight. However, there is no physical battery model in the simulation environment. Therefore, we analyze the "Actuator Outputs (Main)" data in the simulated flight control to estimate the power consumption of the drone during task execution. As shown in the top picture of Fig. 14, each curve corresponds to the PWM control signal of a main motor. Since the simulation does not model the physical loss of the motor, the PWM value is considered to represent the motor speed. According to the empirical physical formula, under the condition of constant air resistance and load, the propeller power P and the motor speed n have a cubic relationship:

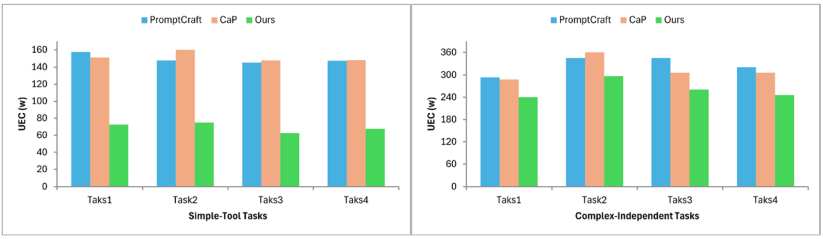


Fig. 13 The performance comparison of UEC with different tasks and HUI frameworks

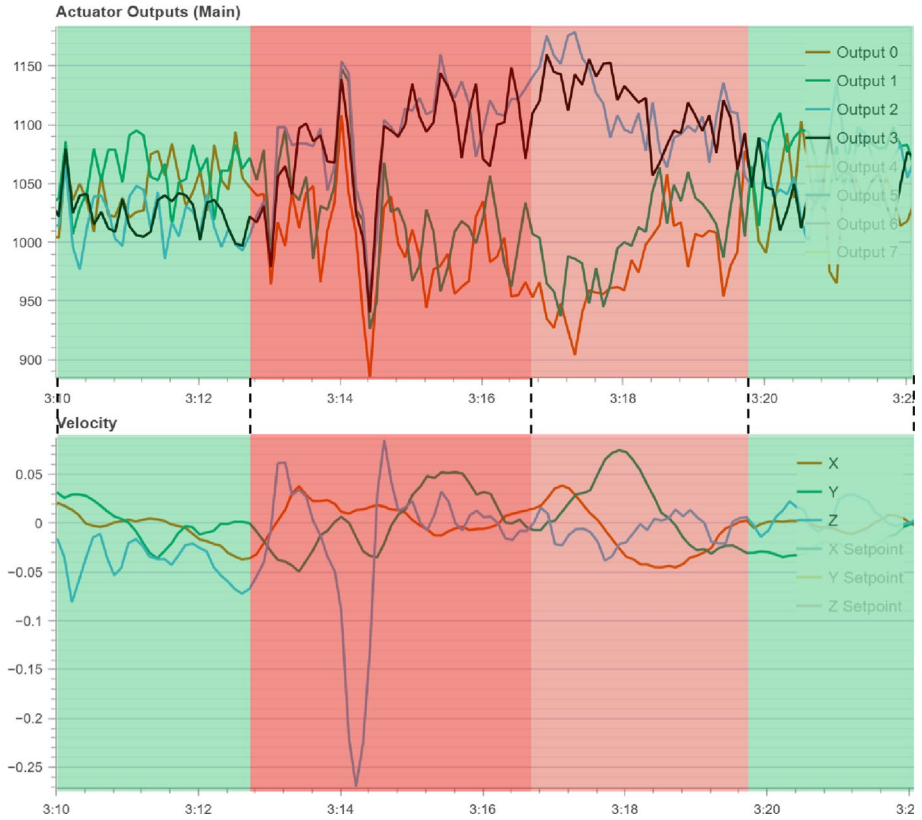


Fig. 14 In the chart, red areas represent the category of low-efficiency flights (low SPR values), and green areas represent the category of high-efficiency flights (high SPR values); different color depths are used to reflect the efficiency differences within the same category — the darker the red, the lower the flight efficiency

$$P \propto n^3 \tag{4}$$

thus, we can obtain the total power:

$$P_{\text{total}}(t) = \sum_{i=1}^{4} P_i(t) \tag{5}$$

finally, we will get the full power during one task:

$$E_{\text{total}} \approx \int P_{\text{total}}(t)dt \tag{6}$$

To more intuitively evaluate the energy consumption of the UAV during task execution, we introduced the “Speed-to-Power Ratio (SPR)" as a visualization indicator on the basis of quantitatively calculating the UEC. This indicator is derived from the average vector veloc-

ity and quantized power consumption within each 2-second interval in the task scenario, and can clearly reflect whether the UAV's power consumption is used to increase speed rather than generating additional losses. We can calculate the SPR through this equation:

$$\mathrm{SPR}_k = \frac{\|\vec{v}_k\|}{\sum_{t=t_k}^{t_k+2s} P_{\mathrm{total}}(t) \cdot \delta t} \tag{7}$$

Where k represents the k-th 2-second time interval, $\vec{v}_k$ is the average velocity vector within this interval, and $P_{\mathrm{total}}(t)$ is the PWM-estimated power within this period of time.

Compared to PromptCraft and CaP, our method achieved an average performance advantage of 70w per task in ST tasks and 55w per task in CI tasks. This advantage stems from the integration of tool invocation and planning ends, enabling LLMs to execute obstacle avoidance tasks in a more direct manner and plan paths from a global perspective without considering the details of specific execution codes.

To further analyze the efficiency changes during flight, we simultaneously monitored the effective operating range of the motor PWM and observed its fluctuation characteristics in the time series. In the Fig. 14, the green area represents stable PWM output and balanced speed, indicating that the aircraft is in a stable propulsion state with high system operating efficiency; the red area shows sharp changes and high-frequency disturbances in the PWM signal, accompanied by speed jumps, which means the system is frequently adjusting the motor to maintain attitude or counter external disturbances, thereby leading to reduced efficiency and increased energy consumption. Based on the SPR value calculated from the vector velocity and quantized power consumption, we visualized the flight path in green (high efficiency) and red (low efficiency), We designed an interior space with three bedrooms, two living rooms, one kitchen and two bathrooms in Fig. 15. Next, we used two ST tasks to demonstrate the visualization of flight efficiency represented by SPR values, as shown in the Figs. 16 and 17:

It is not difficult to find from the quantitative and visualized data that our agent system has achieved a significant improvement in the execution efficiency of ST and CI tasks. First, in terms of the planning and execution of CI tasks, the target sequence of high-level task planning is generated by large models. Since this paper compares a variety of different large model agents, and in order to ensure the fairness of the comparison and in line with the research direction of this paper, we have not carried out special tuning for a single large model in the comparison. It can be seen from the quantitative data that compared with the traditional single-end agent framework, the dual-end agent structure we adopted (which separates the top-level planning and bottom-level execution tasks) shows obvious advantages in the planning of task objectives. Second, it can be clearly seen from the Figs. 16 and 17 that in the ST task, compared with the case where only the large model at the single agent end is used for execution, the waypoints released by our framework during execution have better power efficiency. This smoother waypoint release effect is precisely due to the support of the dedicated tools we used.

We also evaluated the impact of different LLMs on the quality of work produced by this framework Table 3. The models we selected include ERNIE-4.0, GPT-4o, GPT-3, and Liama3 70B. Using the prompt mode of EIP, we extracted 10 tasks from each of the four task categories in the dataset, totaling 40 tasks for classification testing. Table 3 shows the impact of different LLMs models on classification success rates: ERNIE-4.0 and GPT-4o performed the best, followed by Liama3 70B, and GPT-3 performed the worst.



Fig. 15 Simulated indoor environment

4.4 User study

To evaluate the usability and user experience of the UAV-GPT framework in real HUI, we designed and conducted a user survey, inviting a total of 60 participants with professional backgrounds in drone or robot applications to take part in the experiment. We firstly aim to evaluate the user experience of the LLM-based UAV-GPT against the traditional ground station (GS) and secondly aim to compare our dual-agent framework with existing single-agent LLM frameworks (CaP and PromptCraft). To achieve this, we evaluated these two aspects separately which allow us to analyze the specific improvements of our dual-agent architec-

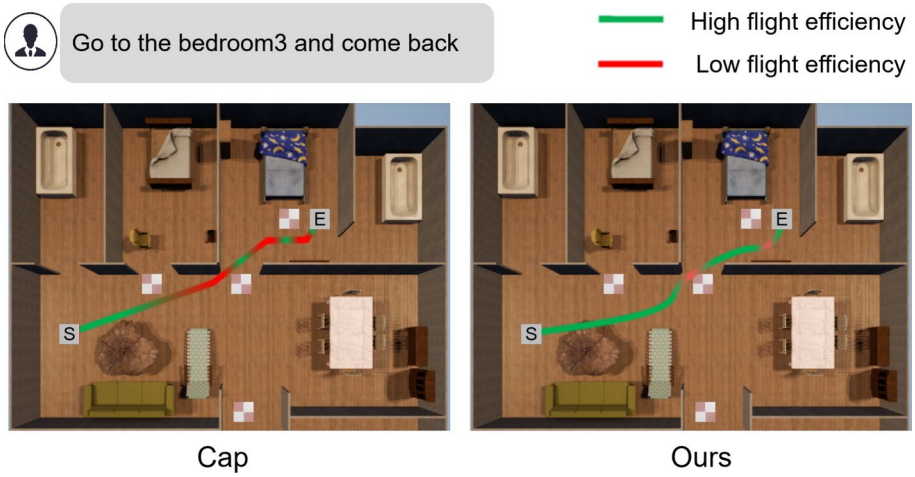


Fig. 16 Simple tool task 1

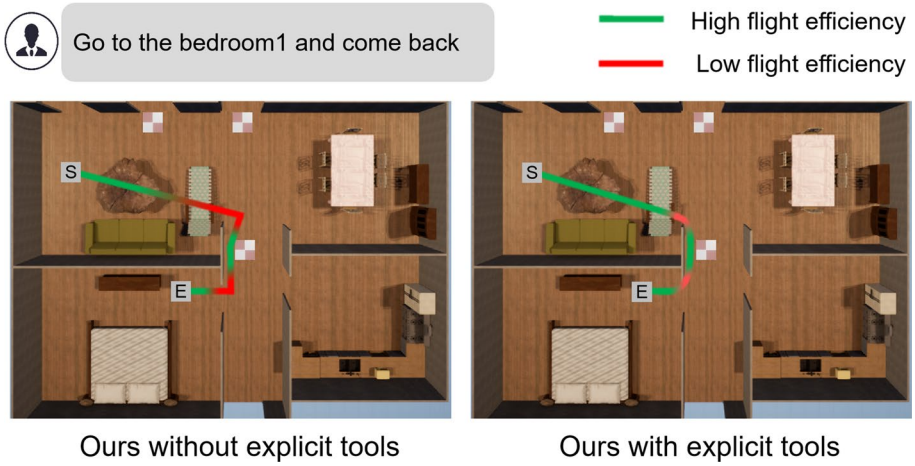


Fig. 17 Simple tool task 2

Table 3 The impact of different LLMs on the quality of work

	ERNIE-4.0	GPT-4o	Llama3 70B	GPT-3
Accuracy rate.	100%	97%	96%	92%

ture over single-agent methods, without the results being confounded by the fundamentally different manual control paradigm.

We firstly constructed a user-study dataset by selecting five representative tasks from each of the four task classes in Section 3.1 (SI, ST, CI, CT), covering scenarios such as target inspection, area patrol and multi-step task planning, resulting in 20 tasks in total. For each chosen task, we generated four execution clips, each showing the task completed by: (1) GS, (2) UAV-GPT, (3) CaP and (4) PromptCraft. Participants were divided into two

groups: Group 1 (n=20) was assigned to evaluate **Q1**, where they selected their preferred methods between UAV-GPT and GS, Group 2 (n=40) was assigned to evaluate the three LLM-based frameworks with respect to **Q2**: overall preference, **Q3**: which is easier to use, **Q4**: which is more adaptable to diverse environments, and **Q5**: which they would prefer in their professional domain. In both groups, participants viewed anonymized execution clips for 5 randomly chosen tasks from user-study dataset. Nest, we analyze the response data of user study.

As shown in Table 4, participants in Group 1 preferred the UAV-GPT over the GS across three task categories, demonstrating advantage in overall user experience. However, the system's reliance on LLM APIs introduces communication and inference latencies compared to direct manual control. This delay is particularly pronounced in complex tasks involving long action sequences, which partially attenuated the user preference for UAV-GPT in such scenarios. Furthermore, as indicated in Table 5, our method consistently outperformed CaP and PromptCraft across all four task categories in terms of **Q2**, **Q3**, **Q4** and **Q5**. The advantage narrowed slightly only in the lower-difficulty SI scenarios, where the selection proportions among the three methods were relatively closer.

To statistically validate the user preferences observed in **Q2** (overall preference among LLM-based frameworks), we analyzed the original data from the 40 participants in Group 2. Each participant completed 5 trials, yielding 200 total observations. For the analysis, we quantified user preference by computing the selection proportion of each method (UAV-GPT, CaP, PromptCraft) for every participant in **Q2**. These proportions served as the dependent variable for a one-way repeated-measures ANOVA with *method* as the within-subject factor. The analysis revealed a significant main effect of the framework method on user preference ($F(2, 78) = 13.12, p < 0.001$). Post-hoc pairwise comparisons indicated that UAV-GPT was chosen significantly more frequently than PromptCraft ($p < 0.001$) and showed a good preference advantage over CaP ($p \approx 0.05$). Additionally, CaP was selected more often than PromptCraft ($p < 0.05$). These statistical findings corroborate the descriptive distributions shown in Table 5, confirming that participants overall prefered UAV-GPT framework over other two methods. To provide a more intuitive presentation of the experimental results, we converted Table 5 into a pie chart as shown in Fig. 18.

Table 4 Preference between UAV-GPT and GS in Group 1 (Q1)

	CT	CI	ST	SI
GS	13	12	9	11
UAV-GPT	11	14	16	14

Table 5 User preferences for LLM-based frameworks in Group 2 (counts of choices)

Method	CT	CI	ST	SI	Q3	Q4	Q5
UAV-GPT	24	23	27	21	120	130	140
CaP	16	23	10	16	20	40	30
PromptCraft	7	5	12	16	60	30	30

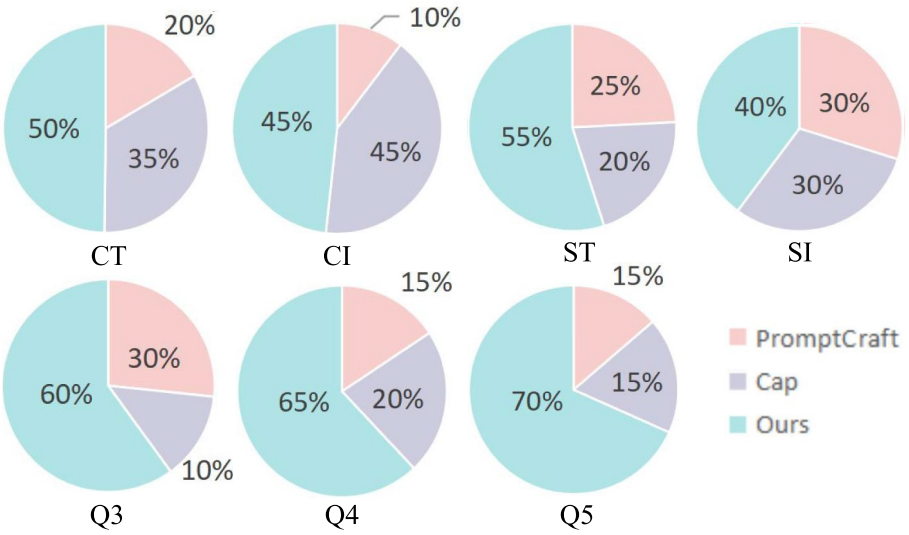


Fig. 18 Userstudy, this study clearly demonstrates that our method is more favored by users compared to the single-agent approach without external tools

5 Discussion and limitations

While UAV-GPT demonstrates significant improvements in task success rates (45.5% ESR gain) and energy efficiency (62.5w UEC reduction per task) compared to single-agent frameworks, several limitations warrant discussion. The parameter tuning process for task complexity coefficients (α, β, γ_p, γ_d, γ_a) relies on empirical fitting from expert-annotated datasets (CLAD and BRMData). Although effective in controlled scenarios Section 4.3, this approach may require manual recalibration when deployed in novel dynamic environments (e.g., agricultural fields with shifting wind patterns), potentially increasing deployment overhead. Regarding tasks adaptability, our framework achieves 96% success in obstacle avoidance (Fig. 11) by integrating real-time replanning tools like EgoPlanner. However, current validation is confined to indoor/static settings (Fig. 8). Extreme outdoor conditions—such as simultaneous multi-obstacle avoidance or strong electromagnetic interference—remain unaddressed. The dependency on ROS-based tools (Section 3.3) further limits applicability to resource-constrained edge devices, a critical gap for field applications like logistics or environmental monitoring.

Future work will prioritize two directions: First, implementing federated learning for autonomous parameter tuning to reduce manual intervention. Second, extending validation to heterogeneous outdoor scenarios with lightweight toolchains.

6 Conclusion

This paper proposed the “UAV-GPT" framework based on the UAV, aiming to achieve more natural and flexible HUI through LLMs. Traditional UAV interaction designs are limited by predefined task planning, making it difficult to meet users’ personalized needs. UAV-GPT achieves direct conversion and execution of user voice or text requests by constructing two LLM agents: a task planning agent and an execution agent. This framework leveraged the natural language understanding capabilities of LLMs to accurately classify and plan complex tasks, intelligently selecting execution codes or invoking tools to tackle them. Experimental results show that UAV-GPT significantly enhanced the fluency of HUI and the flexibility of task execution. Compared to traditional single-ended LLM planning HUI methods, UAV-GPT achieved an average improvement of 24% in Intent Recognition Accuracy, 45.5% in Task Execution Success Rate, and a reduction of 62.5w in energy consumption per task, effectively meeting users’ personalized demands.

Acknowledgements This paper was supported in part by the Zhejiang Provincial Science and Technology Innovation Key Project, China (2023R5211), the National Natural Science Foundation of China (62401506), the Zhejiang Provincial Key Research and Development Program, China (2025C02038, 2025C01SA100286, 2026C02A1244), and the Central Government Guiding Funds for Local Science and Technology Development, China (2025ZY01069, 2026ZY01018).

Author Contributions H.W. and Z.C. wrote the main manuscript text, G.L. conducted the primary simulations and real experiments, B.M. was responsible for reviewing the manuscript and provided numerous suggestions for revision, C.L. was in charge of providing the necessary facilities including but not limited to computers, drones, network, and venues. All authors reviewed and revised the final draft

Data Availability No datasets were generated or analysed during the current study.

Declarations

Competing Interests The authors declare no competing interests.

References

1. Höhrová, P., Soviar, J., & Sroka, W. (2023). Market analysis of drones for civil use. *LOGI: Scientific Journal on Transport and Logistics, 14*(1), 55–65.
2. Flight Standards Department of Civil Aviation Administration of China (2023). Report on Data Statistics of Civil Unmanned Aerial Vehicle Operators and Cloud Systems (2023). http://www.caacnews.com.cn/1/1/202404/W020240426699510487229.pdf
3. Mirri, S., Prandi, C., & Salomoni, P. (2019). Human-Drone Interaction: state of the art, open issues and challenges. In *Proceedings of the ACM SIGCOMM 2019 Workshop on Mobile AirGround Edge Computing, Systems, Networks, and Applications,* pp. 43–48.
4. Kassab, M. A., Ahmed, M., Maher, A., & Zhang, B. (2020). Real-time human-UAV interaction: New dataset and two novel gesture-based interacting systems. *IEEE Access, 8*, 195030–195045.
5. Maher, A., Li, C., Hu, H., & Zhang, B. (2017). Realtime human-UAV interaction using deep learning. In *Biometric Recognition: 12th Chinese Conference, CCBR 2017,* Shenzhen, China, October 28-29, 2017, Proceedings 12, pp. 511–519. Springer.
6. Rajappa, S., Bülthoff, H., & Stegagno, P. (2017). Design and implementation of a novel architecture for physical human-UAV interaction. *The International Journal of Robotics Research, 36*(5–7), 800–819.
7. Haddadin, S., & Croft, E. (2016). Physical human–robot interaction. Springer handbook of robotics, pp. 1835–1874.

8. De Santis, A., Siciliano, B., De Luca, A., & Bicchi, A. (2008). An atlas of physical human-robot interaction. *Mechanism and Machine Theory, 43*(3), 253–270.
9. Castrillo, V. U., Manco, A., Pascarella, D., & Gigante, G. (2022). A review of counter-UAS technologies for cooperative defensive teams of drones. *Drones, 6*(3), 65.
10. Dong, Z., Tan, F., Yu, M., Xiong, Y., & Li, Z. (2024). A bio-inspired sliding mode method for autonomous cooperative formation control of underactuated USVs with ocean environment disturbances. *Journal of Marine Science and Engineering, 12*(9), 1607.
11. Obrenovic, B., Gu, X., Wang, G., Godinic, D., & Jakhongirov, I. (2025). Generative AI and human-robot interaction: implications and future agenda for business, society and ethics. *AI & society, 40*(2), 677–690.
12. Apraiz, A., Lasa, G., & Mazmela, M. (2023). Evaluation of user experience in human-robot interaction: a systematic literature review. *International Journal of Social Robotics, 15*(2), 187–210.
13. Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. (2023). Llama: Open and efficient foundation language models. arXiv:2302.13971
14. Singh, I., Blukis, V., Mousavian, A., Goyal, A., Xu, D., Tremblay, J., Fox, D., Thomason, J., & Garg, A. (2023). PROGPROMPT: program generation for situated robot task planning using large language models. *Autonomous Robots, 47*(8), 999–1012.
15. Vemprala, S.H., Bonatti, R., Bucker, A., & Kapoor, A. (2024). ChatGPT for robotics: Design principles and model abilities. *IEEE Access*.
16. Liu, J., Chai, C., Li, H., Gao, Y., & Zhu, X. (2024). LLM-informed drone visual inspection of infrastructure.
17. Sun, G., Xie, W., Niyato, D., Du, H., Kang, J., Wu, J., Sun, S., & Zhang, P. (2024). Generative AI for advanced UAV networking. arXiv:2404.10556
18. Kim, Y. S., Lee, J., Lee, S., & Kim, M. (2005). A force reflected exoskeleton-type masterarm for human-robot interaction. *IEEE Transactions on Systems, Man, and Cybernetics-Part A: Systems and Humans, 35*(2), 198–212.
19. Scheggi, S., Chinello, F., & Prattichizzo, D. (2012). Vibrotactile haptic feedback for human-robot interaction in leader-follower tasks. In *Proceedings of the 5th International Conference on PErvasive Technologies Related to Assistive Environments,* pp. 1–4.
20. Peppoloni, L., Brizzi, F., Avizzano, C.A., & Ruffaldi, E. (2015). Immersive ROS-integrated framework for robot teleoperation. In *2015 IEEE Symposium on 3D User Interfaces (3DUI),* pp. 177–178. IEEE
21. Tsetserukou, D., Tadakuma, R., Kajimoto, H., Kawakami, N., & Tachi, S. (2007). Towards safe human-robot interaction: Joint impedance control of a new teleoperated robot arm. In: *RO-MAN 2007-The 16th IEEE International Symposium on Robot and Human Interactive Communication,* pp. 860–865. IEEE.
22. Su, Y.-P., Chen, X.-Q., Zhou, T., Pretty, C., & Chase, G. (2022). Mixed-reality-enhanced human-robot interaction with an imitation-based mapping approach for intuitive teleoperation of a robotic arm-hand system. *Applied Sciences, 12*(9), 4740.
23. Fani, S., Ciotti, S., Catalano, M. G., Grioli, G., Tognetti, A., Valenza, G., Ajoudani, A., & Bianchi, M. (2018). Simplifying telerobotics: Wearability and teleimpedance improves human-robot interactions in teleoperation. *IEEE Robotics & Automation Magazine, 25*(1), 77–88.
24. Lahmeri, M.-A., Ghanem, W. R., Bonfert, C., & Schober, R. (2024). Robust trajectory and resource optimization for communication-assisted UAV SAR sensing. *IEEE Open Journal of the Communications Society, 5*, 3212–3228.
25. Arnold, K. P. (2016). The uav ground control station: Types, components, safety, redundancy, and future applications. *International Journal of Unmanned Systems Engineering., 4*(1), 37.
26. Çintaş, E., Özyer, B., & Şimşek, E. (2020). Vision-based moving UAV tracking by another UAV on low-cost hardware and a new ground control station. *IEEE Access, 8*, 194601–194611.
27. Xiong, T., Liu, F., Liu, H., Ge, J., Li, H., Ding, K., & Li, Q. (2023). Multi-drone optimal mission assignment and 3D path planning for disaster rescue. *Drones, 7*(6), 394.
28. Wang, R., Yang, Z., Zhao, Z., Tong, X., Hong, Z., & Qian, K. (2024). LLM-based robot task planning with exceptional handling for general purpose service robots. arXiv:2405.15646
29. Mai, J., Chen, J., Qian, G., Elhoseiny, M., Ghanem, B., et al. (2023). LLM as a robotic brain: Unifying egocentric memory and control.
30. Mower, C.E., Wan, Y., Yu, H., Grosnit, A., Gonzalez-Billandon, J., Zimmer, M., Wang, J., Zhang, X., Zhao, Y., Zhai, A., et al. (2024). ROS-LLM: A ros framework for embodied AI with task feedback and structured reasoning. arXiv:2406.19741
31. Koubaa, A. (2023). Rosgpt: Next-generation human-robot interaction with ChatGPT and ROS.
32. Javaid, S., Fahim, H., He, B., & Saeed, N. (2024). Large language models for UAVs: Current state and pathways to the future. *IEEE Open Journal of Vehicular Technology*.

33. Cui, J., Liu, G., Wang, H., Yu, Y., & Yang, J. (2024). Tpml: Task planning for multi-UAV system with large language models. In *2024 IEEE 18th International Conference on Control & Automation (ICCA),* pp. 886–891. IEEE.
34. Lykov, A., Karaf, S., Martynov, M., Serpiva, V., Fedoseev, A., Konenkov, M., & Tsetserukou, D. (2024). FlockGPT: Guiding UAV flocking with linguistic orchestration. arXiv:2405.05872
35. Tazir, M.L., Mancas, M., & Dutoit, T. (2023). From words to flight: Integrating openai ChatGPT with PX4/gazebo for natural language-based drone control. In *International Workshop on Computer Science and Engineering*.
36. Zhu, Y., Wang, T., Wang, C., Quan, W., & Tang, M. (2023). Complexity-driven trust dynamics in human-robot interactions: insights from AI-enhanced collaborative engagements. *Applied Sciences, 13*(24), 12989.
37. Sweller, J. (2011). Cognitive load theory. In *Psychology of Learning and Motivation,* vol. 55, pp. 37–76.
38. Verwimp, E., Yang, K., Parisot, S., Hong, L., McDonagh, S., Pérez-Pellitero, E., De Lange, M., & Tuytelaars, T. (2023). Clad: A realistic continual learning benchmark for autonomous driving. *Neural Networks, 161*, 659–669.
39. Zhang, T., Li, D., Li, Y., Zeng, Z., Zhao, L., Sun, L., Chen, Y., Wei, X., Zhan, Y., Li, L., et al. (2024). Empowering embodied manipulation: A bimanual-mobile robot manipulation dataset for household tasks. arXiv:2405.18860
40. Liang, J., Huang, W., Xia, F., Xu, P., Hausman, K., Ichter, B., Florence, P., & Zeng, A. (2023). Code as policies: Language model programs for embodied control. In *2023 IEEE International Conference on Robotics and Automation (ICRA),* pp. 9493–9500. IEEE.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.